Technical Report: AgriFlow Intelligence System Design

Author: Kabelo Modimoeng (KOM2047)

Date: 27 February 2026

Project: End-to-End Automated Agricultural ETL Pipeline

---

1. Executive Summary

AgriFlow Intelligence solves a critical operational challenge for agricultural enterprises: calculating daily profitability by unifying internal farm harvest logs (quantities) with external market pricing data. The internal data exists in disparate CSV formats, while external data arrives via JSON from dynamic price feeds.

This system addresses this by implementing a containerized, robust Extract, Transform, Load (ETL) pipeline orchestrated by Apache Airflow. The final output is an idempotent Data Warehouse modeled in a Star Schema using PostgreSQL, ready for downstream analytical consumption.

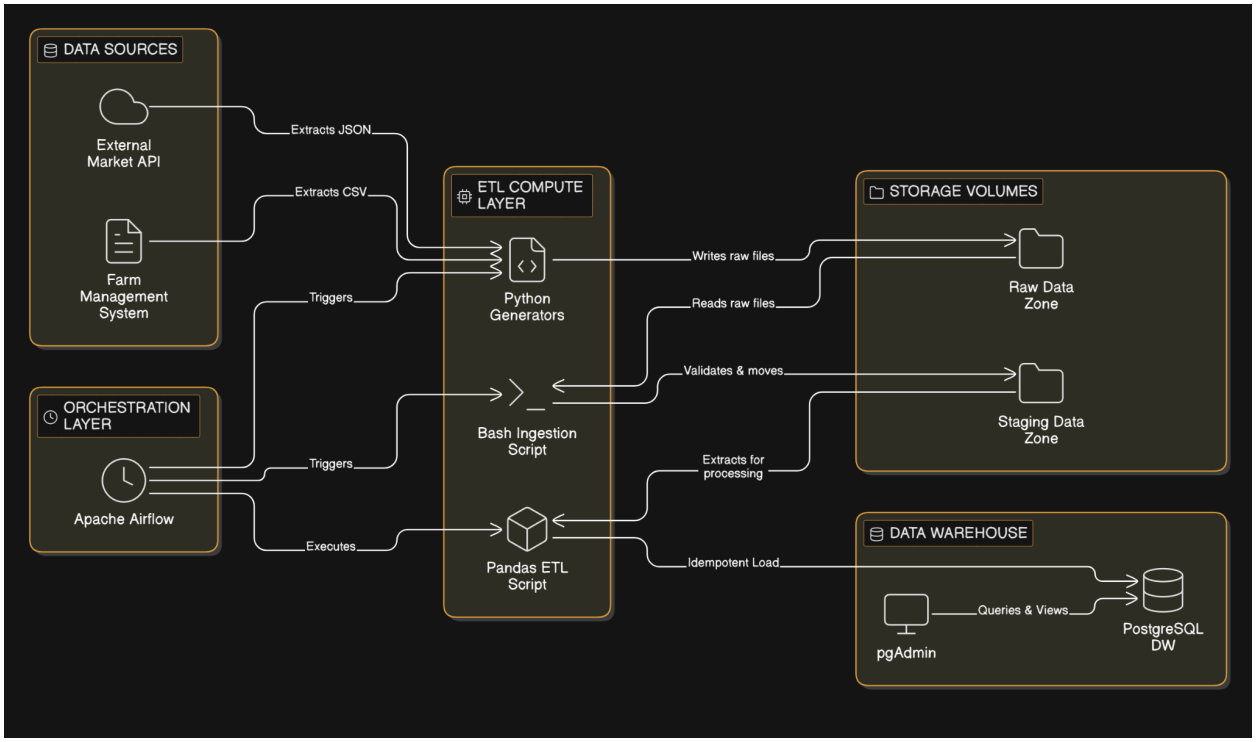
2. Requirements & Business Logic

The system was engineered to meet the following core requirements:

1. **Automation:** The pipeline must run without manual intervention daily.
2. **Data Unification:** internal yields (KG) must be matched against external prices (ZAR per Ton).
3. **Financial Transformations:**
   - **Net Quantity:** qty_harvested_kg - spoilage_kg
   - **Price Conversion:** price_per_ton / 1000.0 (to match yield metrics)
   - **Operational Costs:** Labor costs (fixed R150/hour) and Logistics costs (fixed R2.50/kg transport).
   - **Net Profit:** (Net Quantity * Price Conversion) - (Labor Costs + Logistics Costs)
4. **Idempotency:** Re-running the pipeline for the same date must not create duplicate records or break database integrity.
5. **Containerization:** The entire stack (DB, Airflow, Ingestion) must be portable.

3. System Architecture

The design utilizes a modern, modular Docker-compose architecture separating orchestration, computation, and storage.



Architectural Component Breakdown:

- **Orchestrator (Airflow):** The brain of the system. We use the LocalExecutor within Docker for balance between performance and simplicity. It manages task dependencies, scheduling, and error retries.
- **Staging Area (Data Lake):** A file-based staging zone. We separated data/raw (untrusted data directly from sources) from data/staging (data that has been validated by Bash for correct formats).
- **Transformation Engine (Python/Pandas):** The ETL script (scripts/etl/load.py, leveraging extract.py and transform.py) handles the complex business logic, financial calculations, and joins entirely in-memory using Pandas before loading. This keeps computational load off the operational database.
- **Data Warehouse (PostgreSQL):** The storage layer. We chose a relational model for transactional integrity (ACID compliance) during loading.

4. Data Modeling

The agriflow_dw database utilizes dimensional modeling in a classic **Star Schema**. This design separates business process metrics (Facts) from descriptive attributes (Dimensions), which optimizes OLAP query performance and simplifies analytical reporting.

DDL Structure:

- dim_crop (Dimension): Stores unique crop codes (crop_code), names, and varieties. This table is strictly enforced to ensure clean categorization.

- dim_farm (Dimension): Stores farm IDs, names, and manager information. This design supports Type 2 Slowly Changing Dimensions (SCD Type 2) via is_current, valid_from, and valid_to columns to track changes in farm management over time.
- dim_date (Dimension): A specialized time intelligence dimension where date_id (YYYYMMDD) serves as the joining key, breaking dates down into Day, Month, Year, and Day of Week.
- fact_harvest_yield (Fact): The central metric table. It contains Foreign Keys (crop_id, farm_key, date_id) and all the calculated financial and yield metrics (e.g., profit_zar, logistics_cost_zar). The harvest_id is an identity column but is not used as the functional join key, favoring surrogate keys from dimensions.

5. Technical Design Challenges & Solutions

This project demonstrated robust engineering problem-solving during the implementation and deployment phases.

6. Conclusion

AgriFlow Intelligence provides a production-ready blueprint for automating data unifying pipelines. By leveraging dimensional modeling, containerization, and a focus on idempotent loading, the system provides a reliable, accurate analytical warehouse ready to support operational decision-making.